

Aim : Learn use of matplotlib library as visualisation tool in data science pipeline to create contour plots, histograms and box plots.

Performance : Density and Contour plots Sometimes it is useful to display three-dimensional data in two dimensions using contours or color-coded regions.

There are three Matplotlib functions that can be helpful for this task: `plt.contour` for contour plots, `plt.contourf` for filled contour plots, and `plt.imshow` for showing images.

We will see examples of this type of plot in section below

```
%matplotlib inline
import matplotlib.pyplot as plt
import numpy as np

# Updated seaborn style name (old one gives error)
plt.style.use('seaborn-v0_8-white')
```

```
# A 3D function to plot

def f(x, y):
    return np.sin(x) ** 10 + np.cos(10 + y * x) * np.cos(x)
```

A contour plot can be created with the `plt.contour` function. It takes three arguments: a grid of x values, a grid of y values, and a grid of z values.

The x and y values represent positions on the plot, and the z values will be represented by the contour levels.

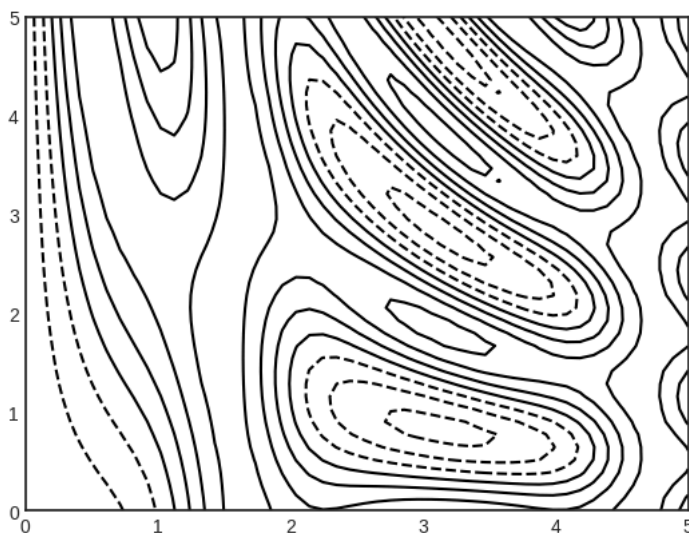
Perhaps the most straightforward way to prepare such data is to use the `np.meshgrid` function, which builds two-dimensional grids from one-dimensional arrays:

```
# x is set of 50 values between 0 and 5
# y is set of 40 values between 0 and 5

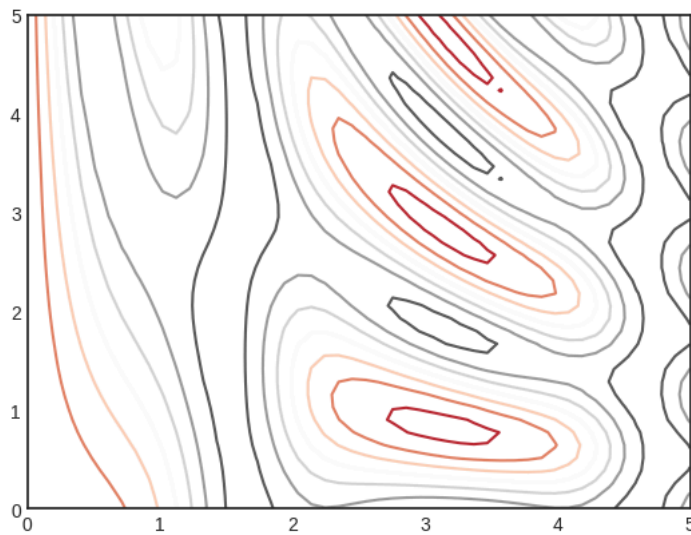
x = np.linspace(0, 5, 50)
y = np.linspace(0, 5, 40)

X, Y = np.meshgrid(x, y)
Z = f(X, Y)
```

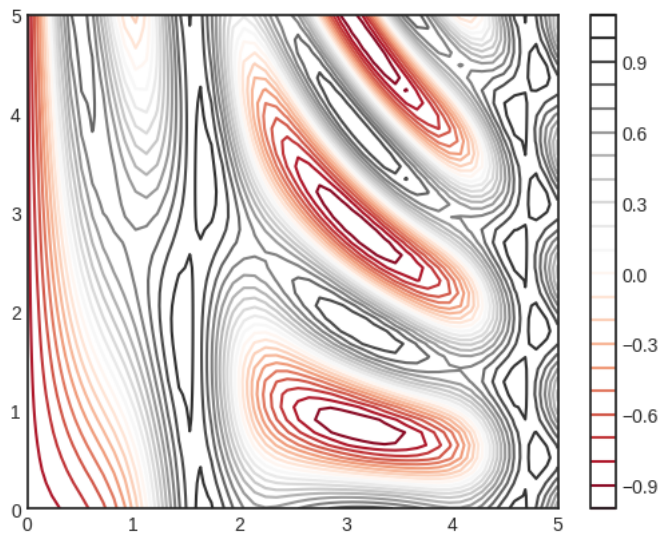
```
plt.contour(X, Y, Z, colors='black');
```



```
plt.contour(X, Y, Z, cmap='RdGy');
```



```
plt.contour(X, Y, Z, 20, cmap='RdGy');
plt.colorbar();
```



Answer Following Questions :

Q1.Explain what are the changes you observe in contour plots above 3 code cells ?

Answer:-detailing the changes observed in the contour plots:

First Plot (plt.contour(X, Y, Z, colors='black')):

Colors: All contour lines are rendered in black. Levels: Matplotlib automatically determines the number of contour levels based on the data's range and distribution. Second Plot (plt.contour(X, Y, Z, cmap='RdGy')):

Colors: This plot uses a colormap named 'RdGy' (Red-Gray). Contour lines are colored according to their Z values, illustrating data intensity. Lighter colors (gray) generally represent lower values, and darker colors (red) represent higher values, or vice-versa depending on the colormap's specifics. Levels: The number of contour levels is still automatically chosen by Matplotlib. Third Plot (plt.contour(X, Y, Z, 20, cmap='RdGy'); plt.colorbar()):

Number of Levels: This plot explicitly specifies 20 contour levels. This significantly increases the detail, showing more distinct changes in the Z values compared to the previous plots. Colors: It continues to use the 'RdGy' colormap, providing color-coded lines based on Z values. Color Bar: A colorbar is added. This provides a legend, indicating what Z value each color in the colormap represents, making the plot much more interpretable.

Q2.What is size of X, Y and Z respectively ?

Answer:-The sizes of X, Y, and Z are all (40, 50).

```
print(f"Shape of X: {X.shape}")
print(f"Shape of Y: {Y.shape}")
print(f"Shape of Z: {Z.shape}")
```

```
Shape of X: (40, 50)
Shape of Y: (40, 50)
Shape of Z: (40, 50)
```

Q3.How does `colorbar()` method help in contour plot.

Answer:-The `colorbar()` method is incredibly helpful in contour plots because it provides a visual legend that maps the colors used in the contour lines or filled regions to their corresponding numerical values. Here's why it's important:

Interpretability: Without a colorbar, it's very difficult to understand what values each color or contour line represents. The colorbar tells you explicitly that, for example, dark red corresponds to high Z values and light gray corresponds to low Z values. **Quantitative Information:** It allows you to quantitatively interpret the plot. You can see the range of values being displayed and roughly estimate the value at any point on the contour plot by looking at its color. **Context:** It provides essential context for the data visualization, especially when using a colormap. Different colormaps can represent data differently, and the colorbar clarifies the specific mapping being used. **Accessibility:** It makes the plot more accessible and understandable to a wider audience, as they don't need prior knowledge of the specific colormap or data range to interpret the plot.

Creating Smoother Contour Plots One potential issue with this plot is that it is a bit "splotchy." That is, the color steps are discrete rather than continuous, which is not always what is desired.

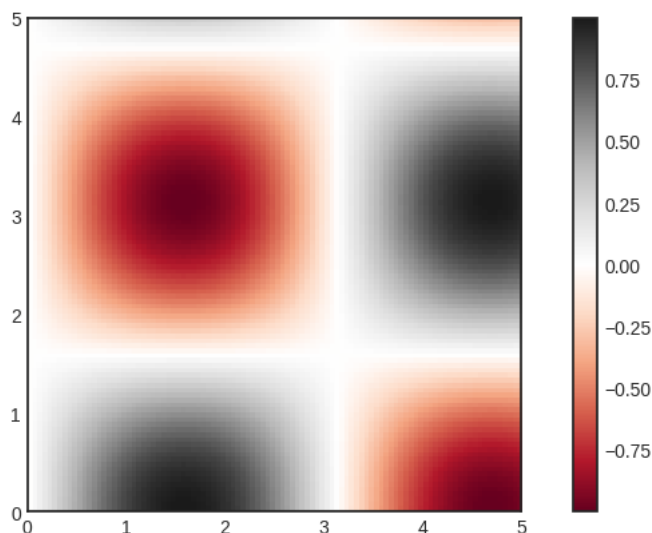
This could be remedied by setting the number of contours to a very high number, but this results in a rather inefficient plot: Matplotlib must render a new polygon for each step in the level. A better way to handle this is to use the `plt.imshow()` function, which interprets a two-dimensional grid of data as an image.

The following code shows how to perform this:

```
import numpy as np
import matplotlib.pyplot as plt

# Create sample data for Z
x = np.linspace(0, 5, 100)
y = np.linspace(0, 5, 100)
X, Y = np.meshgrid(x, y)
Z = np.sin(X) * np.cos(Y)

# Plot image
plt.imshow(Z, extent=[0, 5, 0, 5], origin='lower', cmap='RdGy')
plt.colorbar()
plt.axis('image') # corrected syntax
plt.show()
```



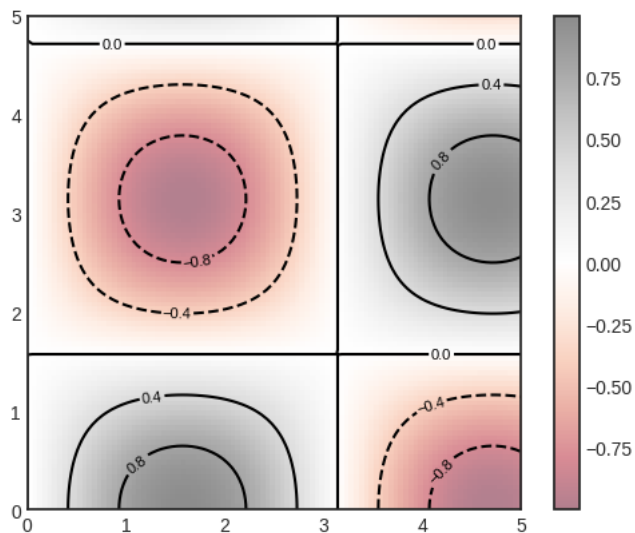
Things to remember while using `imshow()`:

`plt.imshow()` doesn't accept an x and y grid, so you must manually specify the extent `[xmin, xmax, ymin, ymax]` of the image on the plot. `plt.imshow()` by default follows the standard image array definition where the origin is in the upper left, not in the lower left as in most contour plots. This must be changed when showing gridded data. `plt.imshow()` will automatically adjust the axis aspect ratio to match the input data; this can be changed by setting, for example, `plt.axis(aspect='image')` to make x and y units match.

```
contours = plt.contour(X, Y, Z, 5, colors='black')
plt.clabel(contours, inline=True, fontsize=8)

plt.imshow(Z, extent=[0, 5, 0, 5], origin='lower',
```

```
cmap='RdGy', alpha=0.5)
plt.colorbar();
```



Answer Following Questions :

Q1.What does alpha parameter do ?

Answer:-The alpha parameter in plotting functions, like `plt.imshow()`, controls the transparency or opacity of the plotted element. Its value typically ranges from 0 to 1:

alpha = 0: Makes the element completely transparent (invisible). alpha = 1: Makes the element completely opaque (solid). Values between 0 and 1 (e.g., alpha = 0.5): Make the element semi-transparent. This is particularly useful when you want to overlay multiple plots or elements and still be able to see what's underneath. In the context of the `plt.imshow()` call you provided (alpha=0.5), it makes the image 50% transparent, allowing the contour lines plotted beneath it to be partially visible.

What can I help you build? 0 / 2000 Gemini 2.5 Flash Gemini can make mistakes so double-check it and use code with caution. Learn more

Q2.What does clabel method does ?

Answer:-The `clabel` method in Matplotlib is used to add labels to contour lines on a contour plot. These labels typically show the value that each contour line represents, making the plot much more informative and easier to interpret. It takes the contour object as its primary argument. Common parameters include:

`inline`: If True, the labels are drawn within the contour lines, breaking the line where the label is placed. If False, labels are placed next to the contour lines. `fontsize`: Controls the size of the labels. `fmt`: Allows custom formatting of the label text (e.g., to control the number of decimal places). In your code cell EGEh-rK81aQ1, `plt.clabel(contours, inline=True, fontsize=8)` is placing labels directly on the contour lines, showing their corresponding Z-values, with a font size of 8.

Q3.Recreate above code block after, changing parameter 4 in contour fuction to 5 from 3. Explain what change you see in the plot.

Answer:-This will allow you to see the difference directly. Afterwards, I'll explain what changes you observe in the plot.

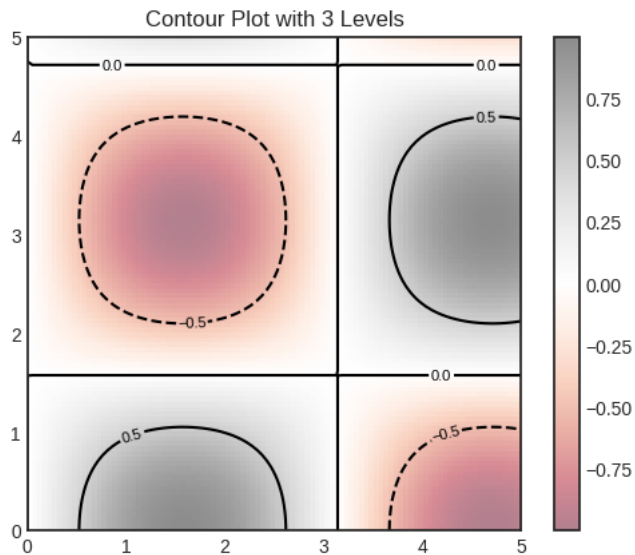
The plots for both 3 and 5 contour levels have been generated successfully! I've also updated the notebook with an explanation of the changes you can observe when adjusting the number of contour levels.

✓ Contour Plot with 3 Levels

```
contours_3 = plt.contour(X, Y, Z, 3, colors='black')
plt.clabel(contours_3, inline=True, fontsize=8)

plt.imshow(Z, extent=[0, 5, 0, 5], origin='lower',
           cmap='RdGy', alpha=0.5)
plt.colorbar();
plt.title('Contour Plot with 3 Levels')
```

Text(0.5, 1.0, 'Contour Plot with 3 Levels')

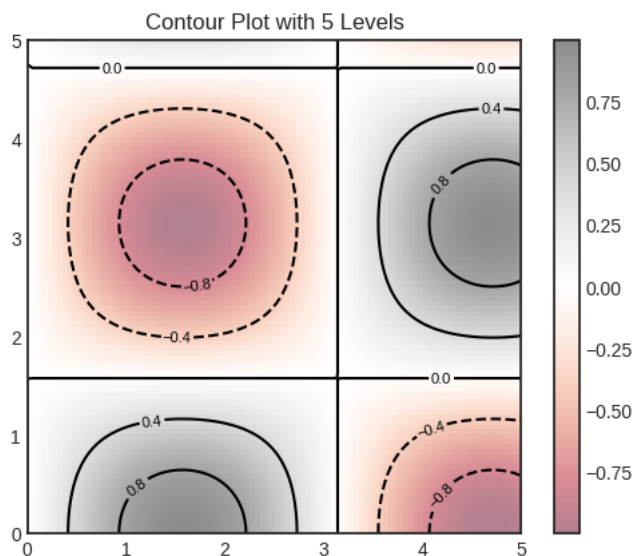


Contour Plot with 5 Levels (Recreation of EGEh-rK81aQ1)

```
contours_5 = plt.contour(X, Y, Z, 5, colors='black')
plt.clabel(contours_5, inline=True, fontsize=8)

plt.imshow(Z, extent=[0, 5, 0, 5], origin='lower',
           cmap='RdGy', alpha=0.5)
plt.colorbar();
plt.title('Contour Plot with 5 Levels')
```

Text(0.5, 1.0, 'Contour Plot with 5 Levels')



Histogram, Binning Plots A histogram is a graphical representation of a grouped frequency distribution with continuous classes. It is an area diagram and can be defined as a set of rectangles with bases along with the intervals between class boundaries and with areas proportional to frequencies in the corresponding classes. In such representations, all the rectangles are adjacent since the base covers the intervals between class boundaries. The heights of rectangles are proportional to corresponding frequencies of similar classes and for different classes, the heights will be proportional to corresponding frequency densities.

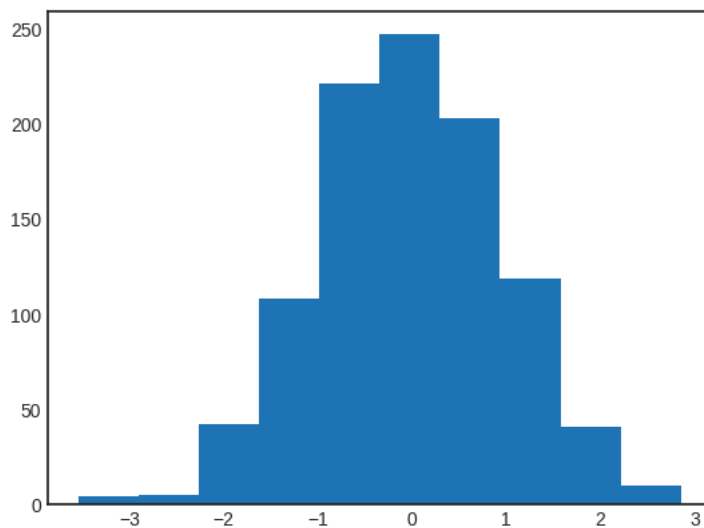
In other words, a histogram is a diagram involving rectangles whose area is proportional to the frequency of a variable and width is equal to the class interval.

The histogram graph is used under certain conditions. They are:

The data should be numerical. A histogram is used to check the shape of the data distribution. Used to check whether the process changes from one period to another. Used to determine whether the output is different when it involves two or more processes. Used to analyse whether the given process meets the customer requirements.

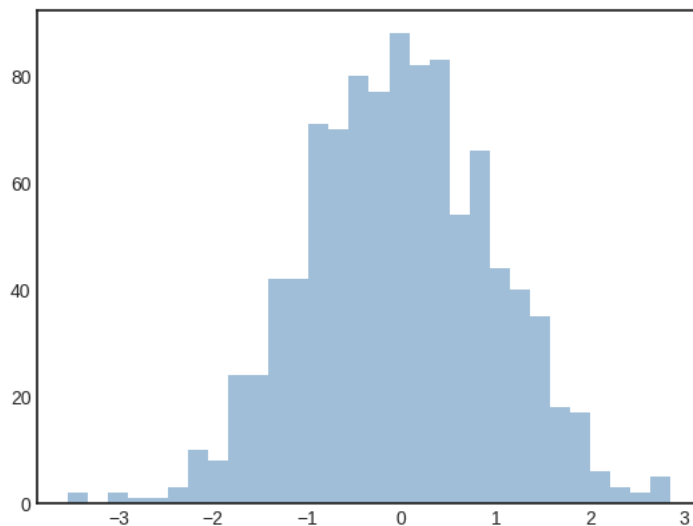
```
# How to plot histogram
```

```
data = np.random.randn(1000)
plt.hist(data);
```



The `hist()` function has many options to tune both the calculation and the display; here's an example of a more customized histogram:

```
plt.hist(data, bins=30, alpha=0.5, histtype='stepfilled', color='steelblue', edgecolor='none');
```

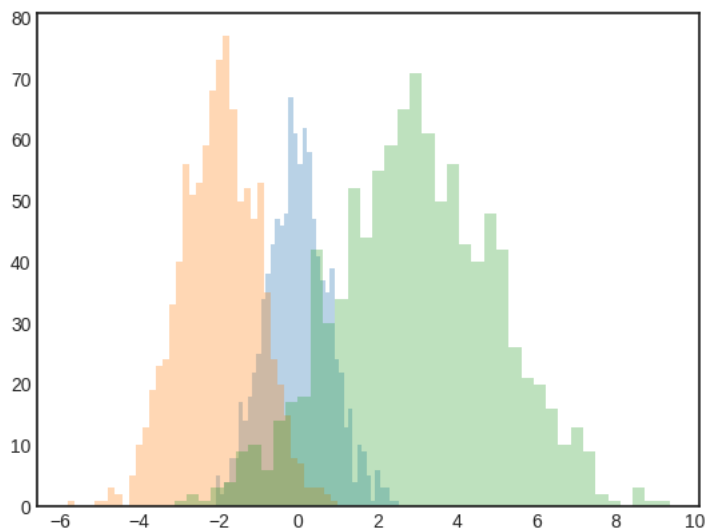


The `plt.hist` docstring has more information on other customization options available. I find this combination of `histtype='stepfilled'` along with some transparency `alpha` to be very useful when comparing histograms of several distributions:

```
x1 = np.random.normal(0, 0.8, 1000)
x2 = np.random.normal(-2, 1, 1000)
x3 = np.random.normal(3, 2, 1000)

kwargs = dict(histtype='stepfilled', alpha=0.3, bins=40)

plt.hist(x1, **kwargs)
plt.hist(x2, **kwargs)
plt.hist(x3, **kwargs);
```



If you would like to simply compute the histogram (that is, count the number of points in a given bin) and not display it, the `np.histogram()` function is available:

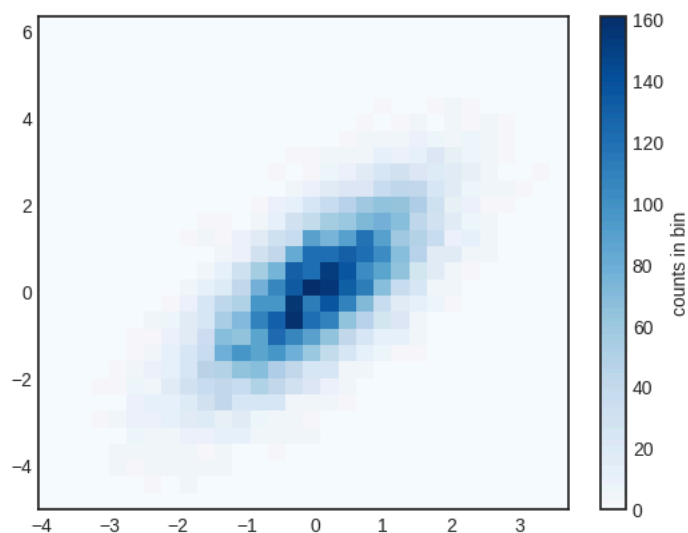
```
#Only storing histogram bin frequencies without plotting it
counts, bin_edges = np.histogram(data, bins=5)
print(counts,bin_edges)
```

```
[ 9 150 468 322 51] [-3.54075358 -2.26240226 -0.98405094  0.29430037  1.57265169  2.85100301]
```

`plt.hist2d`: Two-dimensional histogram One straightforward way to plot a two-dimensional histogram is to use Matplotlib's `plt.hist2d` function:

```
# Create 2 Variable normally distributed data sets with 10000
mean = [0, 0]
cov = [[1, 1], [1, 2]]
x, y = np.random.multivariate_normal(mean, cov, 10000).T
```

```
#Using 2d Histogram plot
plt.hist2d(x, y, bins=30, cmap='Blues')
cb = plt.colorbar()
cb.set_label('counts in bin')
```

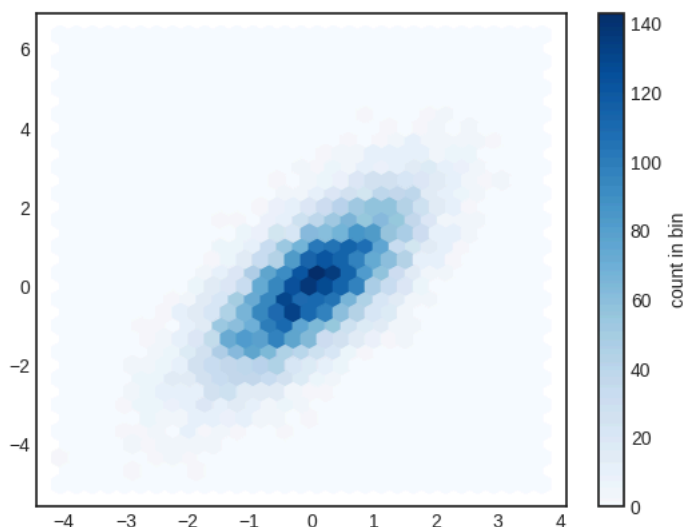


Just as with `plt.hist`, `plt.hist2d` has a number of extra options to fine-tune the plot and the binning, which are nicely outlined in the function docstring. Further, just as `plt.hist` has a counterpart in `np.histogram`, `plt.hist2d` has a counterpart in `np.histogram2d`, which can be used as follows:

```
counts, xedges, yedges = np.histogram2d(x, y, bins=30)
```

plt.hexbin: Hexagonal binnings The two-dimensional histogram creates a tessellation of squares across the axes. Another natural shape for such a tessellation is the regular hexagon. For this purpose, Matplotlib provides the plt.hexbin routine, which will represents a two-dimensional dataset binned within a grid of hexagons:

```
plt.hexbin(x, y, gridsize=30, cmap='Blues')
cb = plt.colorbar(label='count in bin')
```



Questions:

Q1.What is role of bins parameter in histogram plot?

Answer:-The 'bins' parameter in a histogram plot determines the number of intervals, or 'bins', into which the data is divided. These bins are contiguous, non-overlapping intervals that span the range of the data. Here's a breakdown of its role:

Grouping Data: The primary role of bins is to group continuous data into discrete categories. For each bin, the histogram counts how many data points fall within that interval. Shape of the Distribution: The number and width of the bins significantly influence the visual representation of the data's distribution. Too few bins: Can obscure important features of the distribution, making it appear too smooth or generalized. You might miss subtle peaks or gaps in the data. Too many bins: Can make the histogram too noisy or 'spiky', revealing random fluctuations rather than underlying patterns. It might make the distribution look more complex than it actually is.

Interpretability: Choosing an appropriate number of bins is crucial for clearly conveying the underlying distribution of the data. It helps in identifying patterns such as symmetry, skewness, modality (number of peaks), and outliers.

Q2.Create a set of 10000 uniform random samples and store it in variable named norm_data. Draw histograms of norm_data with following bin counts 5, 50, 500, 5000.

Answer:-The code above generates 10,000 uniform random samples and then plots four histograms with bins counts of 5, 50, 500, and 5000 respectively. This visualization clearly demonstrates the effect of the bins parameter:

5 Bins: The histogram appears very coarse, showing a generalized flat distribution, which is expected for uniform data. It hides much of the detail. 50 Bins: This plot provides a more refined view, still showing a relatively flat distribution but with more bars and slightly more visible variations. 500 Bins: As the number of bins increases, the histogram starts to become 'noisy'. While it still generally represents a uniform distribution, individual bars show more erratic fluctuations, making it harder to discern the overall pattern. 5000 Bins: With a very high number of bins, the histogram becomes extremely spiky and sparse. Many bins contain zero or very few data points, making it difficult to interpret the underlying uniform distribution. It highlights the problem of having too many bins, which can introduce visual noise and obscure the true shape of the data.

Generating Uniform Random Samples and Histograms

```
# Create 10,000 uniform random samples
uniform_data = np.random.rand(10000)

# Define the bin counts to use
bin_counts = [5, 50, 500, 5000]

# Create subplots for each histogram
fig, axes = plt.subplots(nrows=2, ncols=2, figsize=(12, 10))
axes = axes.flatten()

for i, bins in enumerate(bin_counts):
```

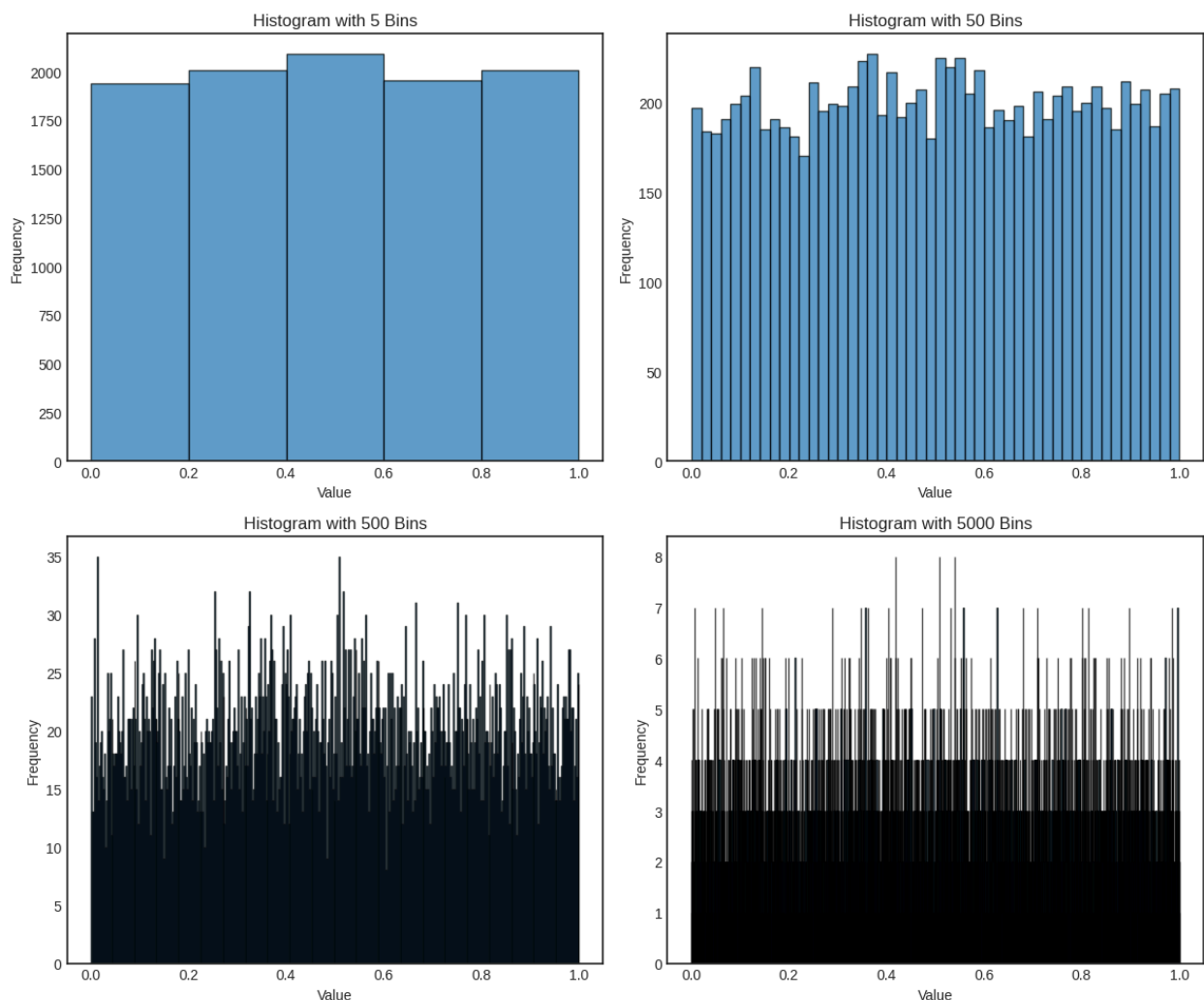


```

ax = axes[i]
ax.hist(uniform_data, bins=bins, edgecolor='black', alpha=0.7)
ax.set_title(f'Histogram with {bins} Bins')
ax.set_xlabel('Value')
ax.set_ylabel('Frequency')

plt.tight_layout()
plt.show()

```



Q3.Explain your observations in text box of above step.

Answer:-the observations from the histogram plots with different bin counts:

The code above generates 10,000 uniform random samples and then plots four histograms with bins counts of 5, 50, 500, and 5000 respectively. This visualization clearly demonstrates the effect of the bins parameter:

5 Bins: The histogram appears very coarse, showing a generalized flat distribution, which is expected for uniform data. It hides much of the detail. **50 Bins:** This plot provides a more refined view, still showing a relatively flat distribution but with more bars and slightly more visible variations. **500 Bins:** As the number of bins increases, the histogram starts to become 'noisy'. While it still generally represents a uniform distribution, individual bars show more erratic fluctuations, making it harder to discern the overall pattern. **5000 Bins:** With a very high number of bins, the histogram becomes extremely spiky and sparse. Many bins contain zero or very few data points, making it difficult to interpret the underlying uniform distribution. It highlights the problem of having too many bins, which can introduce visual noise and obscure the true shape of the data.

Multiple Subplots Sometimes it is helpful to compare different views of data side by side. Matplotlib has the concept of subplots : groups of smaller axes that can exist together within a single figure.

These subplots might be insets, grids of plots, or other more complicated layouts.

```
import matplotlib.pyplot as plt
import numpy as np

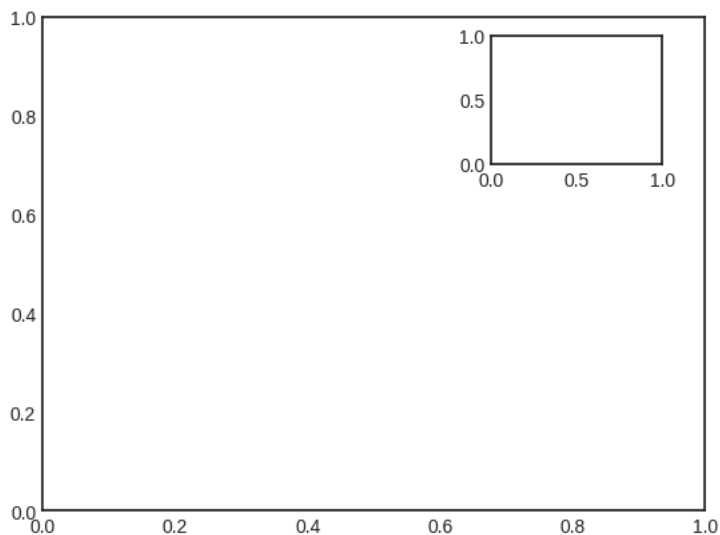
# Updated seaborn style name (new Matplotlib version)
plt.style.use('seaborn-v0_8-white')
```

plt.axes: Subplots by Hand (Coding) The most basic method of creating an axes is to use the `plt.axes` function. By default this creates a standard axes object that fills the entire figure.

`plt.axes` also takes an optional argument that is a list of four numbers in the figure coordinate system. These numbers represent [left, bottom, width, height] in the figure coordinate system, which ranges from 0 at the bottom left of the figure to 1 at the top right of the figure.

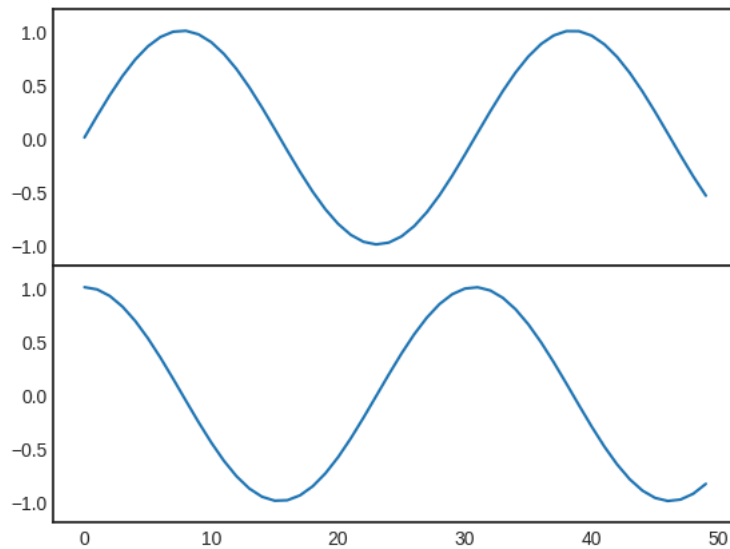
For example, we might create an inset axes at the top-right corner of another axes by setting the x and y position to 0.65 (that is, starting at 65% of the width and 65% of the height of the figure) and the x and y extents to 0.2 (that is, the size of the axes is 20% of the width and 20% of the height of the figure):

```
ax1 = plt.axes() # standard axes
ax2 = plt.axes([0.650, 0.650, 0.2, 0.2]) #inset axes
```



```
#creating subplot using fig.add_axes
fig = plt.figure()
ax1 = fig.add_axes([0.1, 0.5, 0.8, 0.4],
                  xticklabels=[], ylim=(-1.2, 1.2))
ax2 = fig.add_axes([0.1, 0.1, 0.8, 0.4],
                  ylim=(-1.2, 1.2))

x = np.linspace(0, 10)
ax1.plot(np.sin(x))
ax2.plot(np.cos(x));
```

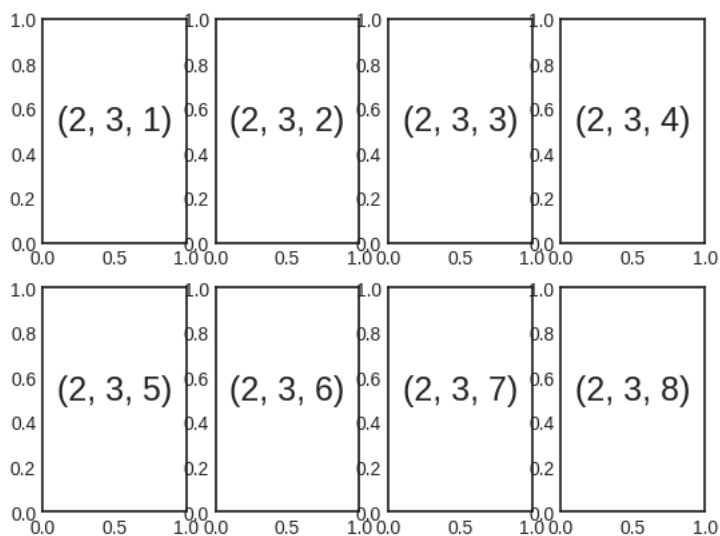


`plt.subplot`: Simple Grids of Subplots Aligned columns or rows of subplots are a common-enough need that Matplotlib has several convenience routines that make them easy to create.

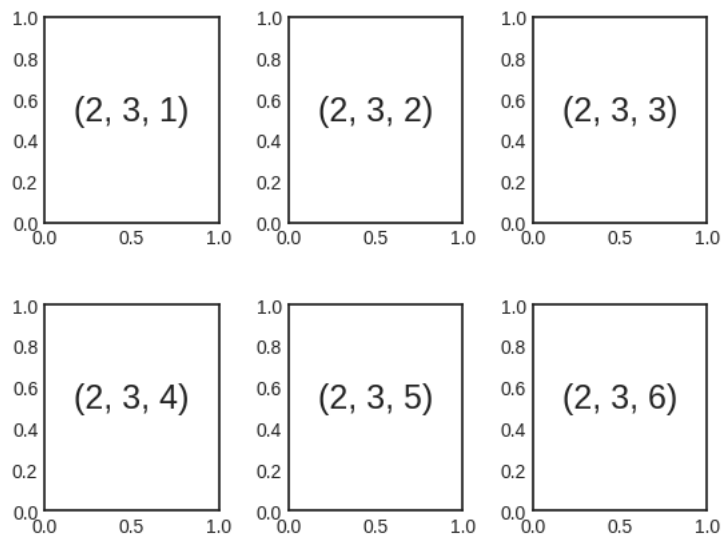
The lowest level of these is `plt.subplot()`, which creates a single subplot within a grid.

As you can see, this command takes three integer arguments—the number of rows, the number of columns, and the index of the plot to be created in this scheme, which runs from the upper left to the bottom right:

```
for i in range(1, 9):
    plt.subplot(2, 4, i)
    plt.text(0.5, 0.5, str((2, 3, i)),
            fontsize=18, ha='center')
```



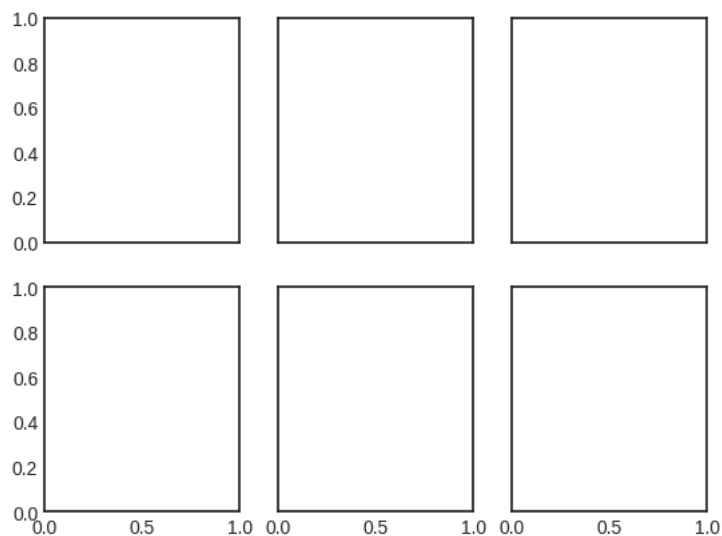
```
fig = plt.figure()
fig.subplots_adjust(hspace=0.4, wspace=0.4)
for i in range(1, 7):
    ax = fig.add_subplot(2, 3, i)
    ax.text(0.5, 0.5, str((2, 3, i)),
           fontsize=18, ha='center')
```



`plt.subplots`: The Whole Grid in One Go The approach just described can become quite tedious when creating a large grid of subplots, especially if you'd like to hide the x- and y-axis labels on the inner plots. For this purpose, `plt.subplots()` is the easier tool to use (note the `s` at the end of `subplots`). Rather than creating a single subplot, this function creates a full grid of subplots in a single line, returning them in a NumPy array. The arguments are the number of rows and number of columns, along with optional keywords `sharex` and `sharey`, which allow you to specify the relationships between different axes.

Here we'll create a grid of subplots, where all axes in the same row share their y-axis scale, and all axes in the same column share their x-axis scale:

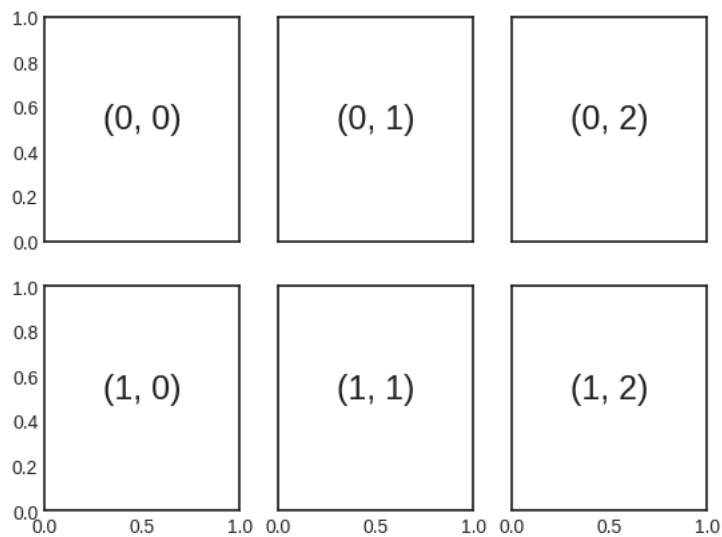
```
fig, ax = plt.subplots(2, 3, sharex='col', sharey='row')
```



Note that by specifying `sharex` and `sharey`, we've automatically removed inner labels on the grid to make the plot cleaner.

The resulting grid of axes instances is returned within a NumPy array, allowing for convenient specification of the desired axes using standard array indexing notation:

```
# axes are in a two-dimensional array, indexed by [row, col]
for i in range(2):
    for j in range(3):
        ax[i, j].text(0.5, 0.5, str((i, j)),
                      fontsize=18, ha='center')
fig
```



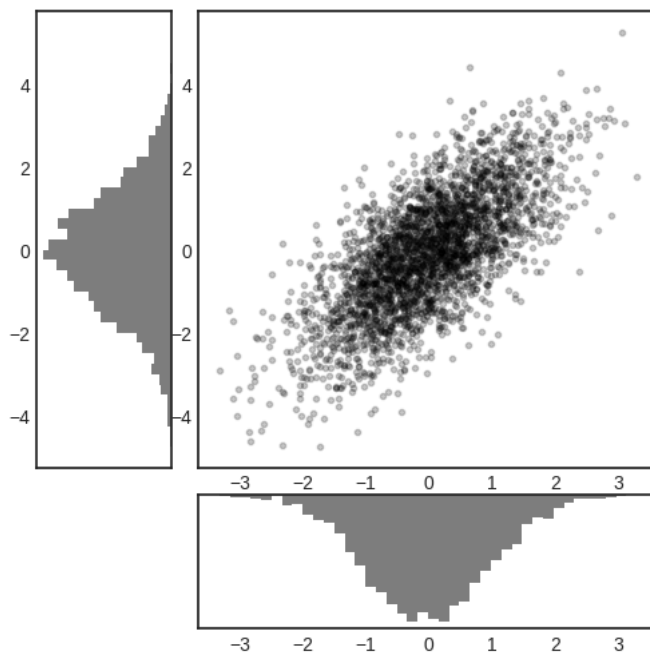
```
# An example of Flexible grid arrangement
# Create some normally distributed data
mean = [0, 0]
cov = [[1, 1], [1, 2]]
x, y = np.random.multivariate_normal(mean, cov, 3000).T

# Set up the axes with gridspec
fig = plt.figure(figsize=(6, 6))
grid = plt.GridSpec(4, 4, hspace=0.2, wspace=0.2)
main_ax = fig.add_subplot(grid[:-1, 1:])
y_hist = fig.add_subplot(grid[:-1, 0], xticklabels=[], sharey=main_ax)
x_hist = fig.add_subplot(grid[-1, 1:], yticklabels=[], sharex=main_ax)

# scatter points on the main axes
main_ax.plot(x, y, 'ok', markersize=3, alpha=0.2)

# histogram on the attached axes
x_hist.hist(x, 40, histtype='stepfilled',
            orientation='vertical', color='gray')
x_hist.invert_yaxis()

y_hist.hist(y, 40, histtype='stepfilled',
            orientation='horizontal', color='gray')
y_hist.invert_xaxis()
```



Boxplot A Box Plot is also known as Whisker plot is created to display the summary of the set of data values having properties like minimum, first quartile, median, third quartile and maximum.

In the box plot, a box is created from the first quartile to the third quartile, a vertical line is also there which goes through the box at the median. Here x-axis denotes the data to be plotted while the y-axis shows the frequency distribution.

```
#Example of Box plot for some random data sets
# Import libraries
import matplotlib.pyplot as plt
import numpy as np

# Creating dataset
np.random.seed(10)

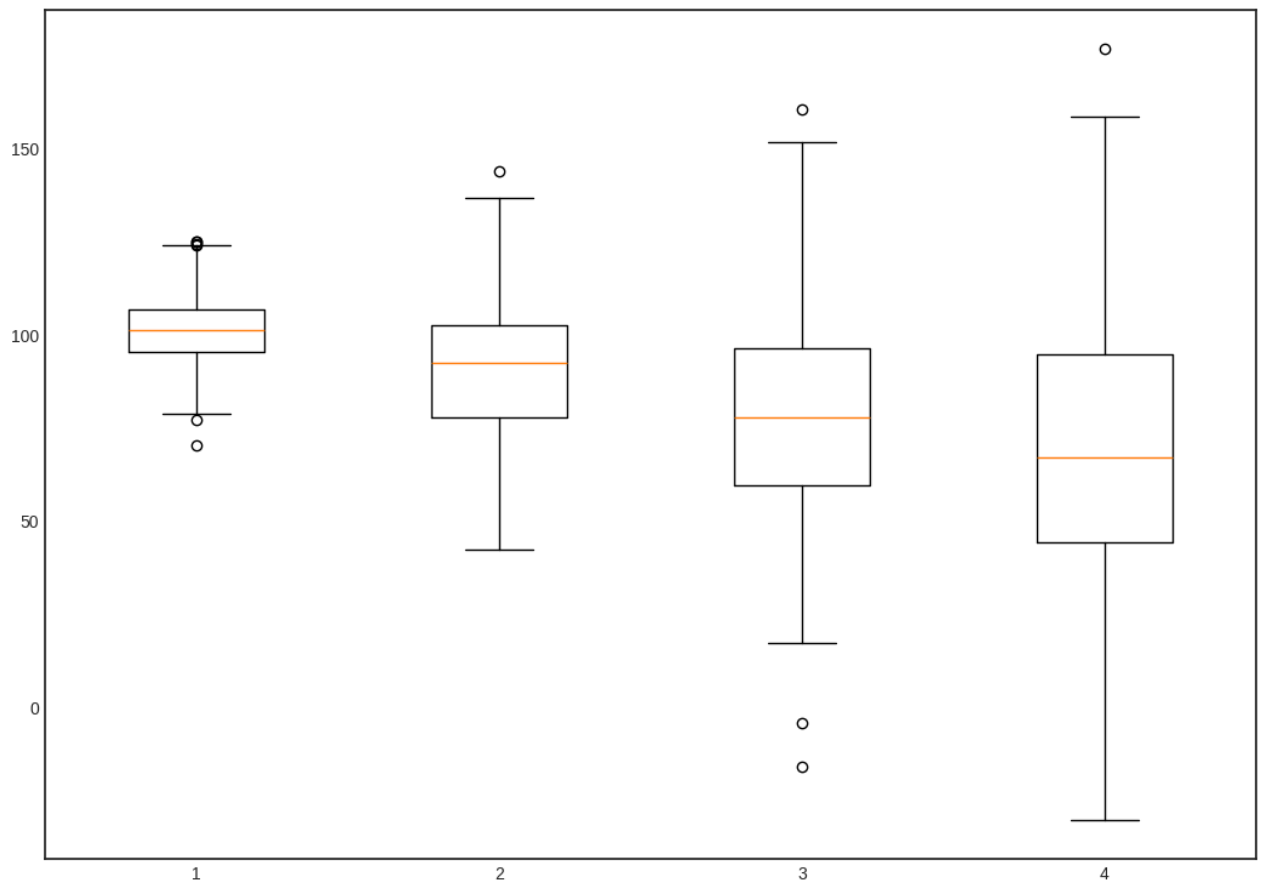
data_1 = np.random.normal(100, 10, 200)
data_2 = np.random.normal(90, 20, 200)
data_3 = np.random.normal(80, 30, 200)
data_4 = np.random.normal(70, 40, 200)
data = [data_1, data_2, data_3, data_4]

fig = plt.figure(figsize =(10, 7))

# Creating axes instance
ax = fig.add_axes([0, 0, 1, 1])

# Creating plot
bp = ax.boxplot(data)

# show plot
plt.show()
```



Questions:

Q1.What is boxplot ?

Answer:-A Box Plot, also known as a Whisker Plot, is a standardized way of displaying the distribution of data based on a five-number summary: minimum, first quartile (Q1), median (Q2), third quartile (Q3), and maximum. It's particularly useful for comparing distributions between groups or for quickly identifying outliers.

Here's a breakdown of what a boxplot shows:

The Box: The central box spans from the first quartile (Q1) to the third quartile (Q3). This represents the Interquartile Range (IQR), which contains the middle 50% of the data. The Median (Q2): A line inside the box marks the median of the data, which is the 50th percentile. The Whiskers: These are lines extending from the box to the minimum and maximum values within a certain range (typically 1.5 times the IQR from the quartiles). They indicate the variability outside the middle 50% of the data. Outliers: Data points that fall outside the whiskers are considered outliers and are often plotted as individual points (e.g., circles or asterisks). What can I help you build? 0 / 2000

Q2.What information do we get from a box plot?